



STARIAX GROUP — European Tech Group

CHALLENGER

Documentation technique

Architecture, infrastructure, modules et exploitation
Application web et mobile native

Référence	CIA-DOC-01
Version	1.0
Date d'émission	19 juillet 2026
Éditeur	STARIAX GROUP — European Tech Group
Produit	Challenger IA (challengeria.fr)
Classification	CONFIDENTIEL — Diffusion restreinte

Ce document contient des informations confidentielles et des secrets d'affaires appartenant à STARIAX GROUP — European Tech Group. Toute reproduction, diffusion ou communication à un tiers, totale ou partielle, est interdite sans autorisation écrite préalable.

1. Mentions légales et clause de confidentialité

1.1 Éditeur et titulaire des droits

STARIAX GROUP (European Tech Group), société éditrice, est titulaire de l'ensemble des droits de propriété intellectuelle portant sur le logiciel Challenger IA, sur sa documentation, sur son architecture, sur ses jeux d'instructions système et sur les marques et logos associés.

Challenger IA est un produit de STARIAX GROUP, exploité sous le nom de domaine challengeria.fr et distribué sous forme d'application web et d'applications mobiles natives iOS et Android (identifiant applicatif tech.stariax.challengeria).

1.2 Objet du document

Le présent document constitue la documentation technique de référence du logiciel Challenger IA : architecture applicative, pile technologique, inventaire des modules, interfaces de programmation, modèle de données, sécurité et procédures d'exploitation.

Le présent document décrit un état du système à la date d'émission indiquée en couverture. Le logiciel étant en développement actif, certaines caractéristiques sont susceptibles d'évoluer. Toute divergence constatée entre ce document et le code source doit être résolue en faveur du code source, qui fait foi.

1.3 Clause de confidentialité

Ce document est classé CONFIDENTIEL. Il contient des informations techniques non publiques, des secrets d'affaires et des éléments constitutifs du savoir-faire de STARIAX GROUP, notamment les jeux d'instructions système, les algorithmes d'analyse cognitive et les mécanismes de notation des sources.

Sa communication est strictement réservée aux personnes ayant besoin d'en connaître dans le cadre de leurs fonctions ou d'une relation contractuelle avec STARIAX GROUP. Toute reproduction, représentation, diffusion, transmission ou mise à disposition d'un tiers, en tout ou partie et par quelque procédé que ce soit, est interdite sans autorisation écrite préalable de l'éditeur.

Le destinataire s'engage à prendre toutes mesures raisonnables pour préserver la confidentialité de ce document, à en restreindre l'accès et à le détruire ou le restituer sur simple demande de l'éditeur.

1.4 Protection au titre du secret des affaires

Les informations contenues dans ce document constituent des secrets d'affaires au sens de l'article L. 151-1 du Code de commerce et de la directive (UE) 2016/943 : elles ne sont pas généralement connues, elles revêtent une valeur commerciale du fait de leur caractère secret, et elles font l'objet de mesures de protection raisonnables de la part de leur détenteur légitime.

Leur obtention, utilisation ou divulgation illicite engage la responsabilité civile de son auteur et peut donner lieu à des mesures provisoires et conservatoires ainsi qu'à des dommages et intérêts.

1.5 Droit applicable et juridiction

Le présent document et les obligations de confidentialité qu'il emporte sont régis par le droit français. Tout litige relatif à son interprétation ou à son exécution relève de la compétence exclusive des tribunaux français compétents, nonobstant pluralité de défendeurs ou appel en garantie.

1.6 Limitation de responsabilité

Ce document est fourni à titre informatif. STARIAX GROUP ne garantit ni l'exhaustivité ni l'absence d'erreur de son contenu et ne saurait être tenu responsable des conséquences d'une utilisation qui en serait faite au-delà de sa finalité documentaire.

Sommaire

1. Mentions légales et clause de confidentialité

2

1.1 Éditeur et titulaire des droits

2

1.2 Objet du document

2

1.3 Clause de confidentialité

2

1.4 Protection au titre du secret des affaires

2

1.5 Droit applicable et juridiction

2

1.6 Limitation de responsabilité

2

2. Présentation générale du système

5

2.1 Caractéristiques du produit

5

2.2 Périmètre fonctionnel

5

3. Architecture applicative

6

3.1 Vue d'ensemble

6

3.2 Principe de sécurité structurant

6

3.3 Pile technologique

6

4. Inventaire des modules applicatifs

8

4.1 Modules principaux

8

4.2 Modules de traitement

8

4.3 Modules d'infrastructure

8

5. Interfaces de programmation serverless

9

5.1 Modules partagés

9

5.2 Points d'entrée

9

5.3 Chaîne de traitement d'une requête de conversation

9

6. Modèle de données

11

6.1 Firestore — contenu utilisateur

11

6.2 Supabase — données transactionnelles

11

6.3 Modèle d'habilitation

11

7. Déploiement et exploitation

13

7.1 Chaîne de construction

13

7.2 Tâches planifiées

13

7.3 Coupure de service pilotée à distance

13

7.4 Applications natives

13

7.5 Variables d'environnement 14

8. Sécurité applicative 15

8.1 Mesures en place 15

8.2 Risques identifiés 15

9. Glossaire technique 16

10. Historique du document 16

2. Présentation générale du système

Challenger IA est une application conversationnelle d'entraînement à la pensée critique. Contrairement à un assistant conversationnel généraliste, dont l'objectif implicite est de satisfaire l'utilisateur, le système est conçu pour le contredire de manière argumentée, vérifier ses affirmations et mesurer dans la durée la qualité de son raisonnement.

Le système repose sur trois piliers techniques distinctifs : un moteur de personas différenciés structurellement, chacun imposant sa propre structure de sortie ; un moteur de vérification factuelle multidimensionnel ; et un profil cognitif longitudinal qui alimente en retour les instructions système des conversations suivantes.

2.1 Caractéristiques du produit

Caractéristique	Valeur
Nom du produit	Challenger IA
Version applicative	1.1.0 (package.json)
Éditeur	STARIAX GROUP — European Tech Group
Domaine public	challengeria.fr
Hébergement de production	Vercel (fonctions serverless et réseau de diffusion)
Identifiant applicatif natif	tech.stariax.challengeria
Plateformes	Web (application web progressive), iOS, Android
Langue de l'interface et des réponses	Français
Volume de code applicatif	16 581 lignes TypeScript (src/) et 1 372 lignes JavaScript (api/)

2.2 Périmètre fonctionnel

- **Chat critique** — cinq personas (Architecte, Fact-Checker, Opposant, Arbitre, Stratège) déclinables sur trois niveaux de friction.
- **Mode débat** — incarnation de trente-cinq personnalités historiques et contemporaines, complétée par un persona personnalisable.
- **Mode entraînement** — douze formats de simulation d'entretien : podcast, entretien d'embauche, oral académique, jurys, présentation, argumentaire.
- **Vérification factuelle** — moteur de deuxième génération avec recherche web intégrée, notation de fiabilité des sources et carte de verdict multidimensionnelle.
- **Profil cognitif** — détection des biais de raisonnement récurrents et des réflexes solides, avec suivi de progression hebdomadaire.
- **Défi quotidien** — sujet renouvelé chaque jour, généré par lot hebdomadaire, récompensé en crédits.
- **Outils partenaires** — six déclinaisons métier : journalisme, éducation, santé, politique, entreprise, création de contenu.
- **Monétisation** — offre gratuite, abonnement Pro et crédits à l'unité, opérés via Stripe.

3. Architecture applicative

3.1 Vue d'ensemble

L'architecture est une application monopage servie statiquement, adossée à des fonctions serverless pour tout traitement nécessitant un secret ou une autorité. Aucun appel à un fournisseur de modèle de langage n'est effectué depuis le navigateur : la clé d'interface de programmation demeure exclusivement côté serveur.

```

NAVIGATEUR / APPLICATION NATIVE (Capacitor)
| React 19 + Vite 6 + Tailwind 4
| Firebase Auth (jeton d'identite) ..... identite
| Firestore ..... conversations, profils
|
| apiFetch() : injecte Authorization: Bearer <jeton>
v
RESEAU DE PERIPHERIE (Vercel Edge)
| middleware.ts ..... coupure de site (STARIAX)
v
FONCTIONS SERVERLESS /api/*
| verifyIdToken() (Firebase Admin) ..... l'uid vient TOUJOURS du jeton
| check_chat_quota() (procedure Supabase) ... debit atomique
|
+--> Mistral AI ..... inference
+--> Tavily ..... recherche web
+--> Supabase ..... credits, quotas, abonnements
+--> Stripe ..... paiement
+--> Resend ..... courriel transactionnel
+--> STARIAX ..... maintenance, beta, erreurs

```

3.2 Principe de sécurité structurant

L'identifiant utilisateur n'est jamais accepté depuis le client

Toutes les fonctions serverless dérivent l'identifiant utilisateur du jeton Firebase vérifié (`api_lib/admin.js`), jamais du corps de la requête. Cette règle rend structurellement impossible la lecture ou la modification du compte d'un tiers par manipulation de la requête. Elle s'applique sans exception aux points d'entrée `/api/chat`, `/api/credits` et `/api/beta`.

3.3 Pile technologique

3.3.1 Dépendances de production

Domaine	Bibliothèque	Version	Rôle
Interface	react / react-dom	^19.0.0	Rendu
Interface	motion	^12.23.24	Animations
Interface	lucide-react	^0.546.0	Iconographie
Interface	react-markdown, remark-gfm	^10.1.0 / ^4.0.1	Rendu des réponses
Construction	vite	^6.2.0	Empaquetage et serveur de développement
Construction	tailwindcss, @tailwindcss/vite	^4.1.14	Styles
Identité	firebase	^11.0.0	Authentification et Firestore côté client
Identité	firebase-admin	^12.7.0	Vérification des jetons côté serveur
Données	@supabase/supabase-js	^2.101.1	Crédits, quotas, abonnements
Paieement	stripe	^22.0.0	Abonnements et crédits
Natif	@capacitor/core, ios, android	^8.4.2	Encapsulation mobile
Natif	@capacitor/share, filesystem	^8.0.1 / ^8.1.2	Partage de verdict
Documents	jspdf	^4.2.1	Export PDF de session
Documents	pdfjs-dist	^5.6.205	Lecture des pièces jointes PDF

Domaine	Bibliothèque	Version	Rôle
Documents	jszip	^3.10.1	Export groupé
Notification	web-push	^3.6.7	Notifications poussées
Mesure	@vercel/analytics	^2.0.1	Audience

3.3.2 Outillage de développement

typescript ~5.8.2, @types/node ^22.14.0, @types/react-dom ^19.2.3. La vérification de types constitue l'unique contrôle automatisé du projet : la commande `npm run lint` exécute `tsc --noEmit`.

Points de vigilance relevés lors de l'inventaire

1) Le fichier `tsconfig.json` ne déclare que le répertoire `src` : les fonctions `serverless (api/*.js)` et `middleware.ts` échappent donc à la vérification de types exécutée en intégration continue. 2) Le paquet `@types/react` n'est pas déclaré dans `package.json`. 3) Aucun test automatisé n'est présent dans le projet. 4) Plusieurs outils de construction (`vite`, `tailwindcss`, `@capacitor/cli`) sont déclarés en dépendances de production plutôt qu'en dépendances de développement.

4. Inventaire des modules applicatifs

Le répertoire `src/` comprend trente-trois fichiers TypeScript pour 16 581 lignes. Le composant racine `App.tsx` en concentre 40 %, ce qui constitue le principal point de dette technique structurelle du projet.

4.1 Modules principaux

Fichier	Lignes	Rôle
<code>src/App.tsx</code>	6 630	Composant racine : chat, authentification, personas, barre latérale, dossiers, pièces jointes, mode vocal, crédits
<code>src/debatePersonas.ts</code>	1 708	Bibliothèque des trente-cinq personas de débat et persona personnalisable
<code>src/SettingsPage.tsx</code>	1 511	Réglages : profil, neuro-atypies, Big Five, abonnement, crédits, export et import
<code>src/outils/JournalismeApp.tsx</code>	1 351	Outil partenaire Journalisme
<code>src/interviewTypes.ts</code>	722	Configuration des douze formats d'entraînement à l'entretien
<code>src/outils/OutilsPage.tsx</code>	701	Portail des outils partenaires
<code>src/LibraryPage.tsx</code>	515	Bibliothèque d'entraînements

4.2 Modules de traitement

Fichier	Lignes	Rôle
<code>src/viz/VizBlocks.tsx</code>	349	Rendu des visuels honnêtes : balance, carte d'argument, verdict
<code>src/viz/vizParse.ts</code>	199	Analyse tolérante des marqueurs de visuel
<code>src/cognitive.ts</code>	229	Profil cognitif : biais, forces, progression
<code>src/pdfExport.ts</code>	316	Export PDF de session
<code>src/markdownExport.ts</code>	191	Export Markdown
<code>src/verdictShare.ts</code>	158	Partage natif d'une carte de verdict
<code>src/dailyChallenges.ts</code>	146	Défis quotidiens et progression
<code>src/sounds.ts</code>	186	Effets sonores générés par synthèse audio, sans aucun fichier son
<code>src/factcheck/SourcesPanel.tsx</code>	118	Panneau de sources et puces de citation

4.3 Modules d'infrastructure

Fichier	Lignes	Rôle
<code>src/stariax.ts</code>	136	Client STARIAX : maintenance, logo, offres, remontée d'erreurs
<code>src/StariaxGate.tsx</code>	107	Neutralisation d'interface côté client, indispensable en natif
<code>src/beta.ts</code>	101	Résolution du statut bêta via le relais serveur
<code>src/BetaBadge.tsx</code>	47	Badge de version bêta
<code>src/supabase.ts</code>	94	Client Supabase navigateur, clé anonyme
<code>src/firebase.ts</code>	81	Initialisation Firebase et persistance de session
<code>src/apiClient.ts</code>	35	Enveloppe de requête : injection du jeton, base d'appel native
<code>src/push.ts</code>	62	Agent de service et abonnement aux notifications
<code>src/userProfile.ts</code>	154	Types et persistance du profil utilisateur
<code>src/ErrorBoundary.tsx</code>	57	Écran de secours et remontée des erreurs de rendu
<code>src/stariaxErrors.ts</code>	32	Capture des erreurs d'exécution non gérées
<code>src/main.tsx</code>	20	Point d'entrée de l'application

5. Interfaces de programmation serverless

Quinze fonctions serverless, écrites en JavaScript, totalisant 1 372 lignes. Trois modules partagés assurent l'authentification, la gestion des origines et le contrôle de quota.

5.1 Modules partagés

Module	Rôle	Variables d'environnement
api/_lib/admin.js	Vérification du jeton Firebase transmis en en-tête d'autorisation. Mode dégradé si non configuré.	FIREBASE_ADMIN_PROJECT_ID, FIREBASE_ADMIN_CLIENT_EMAIL, FIREBASE_ADMIN_PRIVATE_KEY
api/_lib/cors.js	En-têtes d'origine pour l'application native et traitement de la requête préalable.	—
api/_lib/quota.js	Limitation de débit, quota journalier et débit de crédits, de manière atomique.	VITE_SUPABASE_URL, SUPABASE_SERVICE_ROLE_KEY, CHAT_RATE_LIMIT_PÉR_MIN (30), CHAT_FREE_DAILY_LIMIT (20), CHAT_PRO_DAILY_LIMIT (150)

5.2 Points d'entrée

Route	Méthode	Rôle
/api/chat	POST	Relais vers Mistral AI avec diffusion en flux, recherche web automatique et notation des sources. Authentifié et soumis à quota.
/api/credits	GET, POST	Lecture du solde et de l'offre ; réclamation de la récompense quotidienne et débit d'un crédit.
/api/beta	GET	Résolution du statut bêta via STARIAX, sans exposer l'identifiant utilisateur ni le secret produit.
/api/generate-challenges	Planifié	Génération hebdomadaire de sept défis quotidiens, écrits dans Firestore.
/api/reengagement	Planifié	Relance par courriel des utilisateurs inactifs ayant consenti aux communications.
/api/send-reset	POST	Courriel de réinitialisation de mot de passe personnalisé.
/api/stripe-webhook	POST	Réception des événements Stripe, avec vérification de signature sur le corps brut.
/api/push-subscribe	POST	Enregistrement de l'abonnement aux notifications.
/api/push-broadcast	POST	Diffusion d'une notification à l'ensemble des abonnés. Protégé par jeton partagé.
/api/marketing-consent	POST	Enregistrement du consentement aux communications et désabonnement.
/api/search-context	GET	Contexte encyclopédique pour les personas de débat.
/api/reset-debug	—	Diagnostic de configuration du service de courriel. Marqué TEMPORAIRE dans le code.

Points d'entrée nécessitant une révision

api/search-context.js ne comporte ni contrôle d'origine, ni authentification, ni limitation de débit : il relaie des requêtes arbitraires vers des services tiers. api/reset-debug.js porte la mention « TEMPORAIRE — À SUPPRIMER » dans son en-tête, n'est pas authentifié et ne vérifie pas la méthode HTTP ; il expose publiquement l'état de configuration du service d'envoi de courriels. Ces deux points devraient être traités avant une ouverture large du service.

5.3 Chaîne de traitement d'une requête de conversation

La fonction api/chat.js applique la séquence suivante, dans cet ordre :

- Application des en-têtes d'origine et rejet de toute méthode autre que POST.
- Validation du corps de requête : la liste de messages et le modèle sont obligatoires.

- **Refus par défaut en production** — si la vérification de jeton n'est pas active, la fonction répond 503 plutôt que de servir une requête non authentifiée.
- Vérification du jeton Firebase ; en l'absence d'identifiant, réponse 401.
- Calcul du coût : un crédit, majoré de trois crédits par pièce jointe, dans la limite de trois pièces jointes.
- Contrôle atomique de quota. Une limitation de débit renvoie 429, un solde insuffisant renvoie 402.
- Détection du besoin de recherche web, interrogation du moteur de recherche puis notation de fiabilité des sources.
- Appel au modèle et diffusion en flux, précédée d'un événement de métadonnées portant les sources citées.

6. Modèle de données

La persistance est répartie entre trois systèmes selon la nature de la donnée : Firestore pour le contenu utilisateur, Supabase pour les données transactionnelles et de quota, le stockage local du navigateur pour les préférences d'interface.

6.1 Firestore — contenu utilisateur

Chemin	Contenu
users/{uid}/conversations/{id}	Conversation complète : titre, persona, niveau, messages, pièces jointes, sources
users/{uid}/projects/{id}	Dossier de classement : nom, couleur, état replié
users/{uid}/meta/usage	Compteurs d'usage journalier et hebdomadaire
users/{uid}/meta/consent	Preuve de consentement aux conditions générales et horodatage
users/{uid}/meta/profile	Copie serveur du profil utilisateur
users/{uid}/meta/cognitive	Profil cognitif : compteurs de biais, de forces et série hebdomadaire
shared/{shareId}	Conversation partagée par lien. Lecture publique.
daily_challenges/{AAAA-MM-JJ}	Défi du jour. Lecture publique, écriture réservée au serveur.

6.2 Supabase — données transactionnelles

Table	Colonnes principales	Rôle
user_credits	user_id (clé), credits, lifetime_credits	Solde de crédits
subscriptions	user_id (clé), email, stripe_customer_id, plan, status, current_period_end	Abonnement
chat_usage	user_id (clé), day, day_count, minute_window, minute_count	Quota et limitation de débit
daily_rewards	user_id et day (clé composite)	Récompense quotidienne, une par jour
user_contacts	user_id (clé), email, marketing_opt_in, opt_in_at, last_reengaged_at	Consentement aux communications
push_subscriptions	endpoint (clé), user_id, subscription	Abonnement aux notifications
subscribers	email (clé)	Inscriptions à la lettre d'information
daily_challenges_cache	day (clé), payload	Cache des défis

6.2.1 Procédures stockées

- **check_chat_quota** — contrôle atomique combinant la fenêtre de limitation d'une minute, le quota journalier selon l'offre, et la bascule sur les crédits lorsque le quota est épuisé. Retourne un motif normalisé.
- **claim_daily_reward** — attribution de la récompense quotidienne, idempotente par contrainte d'unicité sur le couple utilisateur et jour.
- **add_credits** et **deduct_one_credit** — mouvements de crédits.

6.3 Modèle d'habilitation

La sécurité d'accès repose sur deux mécanismes complémentaires, l'un dans Firestore, l'autre dans Supabase.

- **Firestore** — les documents rattachés à un utilisateur ne sont accessibles qu'à leur propriétaire. Les défis quotidiens sont en lecture publique et en écriture interdite. Une règle terminale refuse tout accès non explicitement autorisé.
- **Supabase** — la sécurité au niveau des lignes est activée sans aucune politique sur les tables sensibles, ce qui équivaut à un refus total pour les rôles anonyme et authentifié. Seul le rôle de service, détenu par le serveur, y accède. Les droits d'exécution des procédures stockées sont révoqués pour ces mêmes rôles.

Anomalie constatée sur le partage de conversation

Les documents de partage sont en lecture publique par conception, ce qui correspond à la fonction de partage par lien. Toutefois, la charge écrite par l'application ne contient pas de champ propriétaire, alors que les règles de modification et de suppression l'exigent. En conséquence, une conversation partagée ne peut être ni modifiée ni supprimée par son auteur. Ce point a une incidence directe sur le droit à l'effacement et doit être corrigé.

7. Déploiement et exploitation

7.1 Chaîne de construction

Élément	Valeur
Commande de construction	npm run build
Répertoire de sortie	dist
Cadre déclaré	vite
Réécritures	Les routes /api/ sont conservées ; toute autre route est servie par index.html
Intégration continue	Node 20, installation verrouillée, vérification de types, construction

7.2 Tâches planifiées

Route	Planification	Objet
/api/generate-challenges	Dimanche, 03h00 UTC	Génération de sept défis quotidiens
/api/reengagement	Lundi, 10h00 UTC	Relance des utilisateurs inactifs

Les deux tâches vérifient un jeton partagé. La fonction de diffusion de notifications est protégée par le même secret mais n'est déclarée dans aucune planification : son déclenchement est manuel.

7.3 Coupure de service pilotée à distance

Le fichier middleware.ts, exécuté sur le réseau de périphérie de l'hébergeur, interroge la plateforme STARIAX à chaque requête afin de déterminer si le produit doit être rendu indisponible. La réponse est mise en cache trente secondes par instance.

- Périmètre : toutes les routes hors interfaces de programmation, ressources statiques et fichiers portant une extension.
- Coupure globale : redirection temporaire vers la page de maintenance hébergée par STARIAX.
- Coupure partielle : réponse 503 accompagnée d'un en-tête de réessai.
- **Comportement en cas de défaillance** — toute erreur réseau ou de traitement laisse passer la requête. La plateforme de pilotage ne peut donc jamais rendre le produit indisponible par sa propre panne.

Limite structurelle du middleware

L'application native chargeant ses ressources depuis une origine locale, elle ne traverse jamais le réseau de périphérie de l'hébergeur. Le middleware est donc sans effet sur iOS et Android. C'est le module src/StariaxGate.tsx, côté client, qui assure la coupure sur ces plateformes.

7.4 Applications natives

Élément	Valeur
Encapsulation	Capacitor 8.4
Identifiant applicatif	tech.stariax.challengeria, cohérent sur iOS, Android et la configuration
Répertoire web	dist
Couleur de fond	#F0F4FF
Particularité iOS	Marge de contenu neutralisée
Synchronisation	npm run cap:sync, cap:ios, cap:android

La persistance de session est initialisée en privilégiant la base de données du navigateur, avec repli sur le stockage local. Cette configuration est nécessaire au fonctionnement de l'authentification dans le conteneur natif, dont l'origine diffère de celle du web.

7.5 Variables d'environnement

Les variables préfixées `VITE_` sont incorporées au paquet client au moment de la construction : elles sont donc publiques et ne doivent contenir aucun secret. Leur modification impose une reconstruction, la seule mise à jour de la valeur étant sans effet sur un paquet déjà construit.

Portée	Variables
Serveur uniquement (secrets)	MISTRAL_API_KEY, TAVILY_API_KEY, SUPABASE_SERVICE_ROLE_KEY, FIREBASE_ADMIN_PROJECT_ID, FIREBASE_ADMIN_CLIENT_EMAIL, FIREBASE_ADMIN_PRIVATE_KEY, STRIPE_SECRET_KEY, STRIPE_WEBHOOK_SECRET, RESEND_API_KEY, VAPID_PRIVATE_KEY, CRON_SECRET, STARIAX_PRODUCT_SECRET
Client (publiques par conception)	VITE_FIREBASE_*, VITE_SUPABASE_URL, VITE_SUPABASE_ANON_KEY, VITE_VAPID_PUBLIC_KEY, VITE_STRIPE_*, VITE_API_BASE, VITE_STARIAX_BASE, VITE_STARIAX_PRODUCT_ID, VITE_STARIAX_INGEST_TOKEN
Serveur, non secrètes	STARIAX_BASE, STARIAX_PRODUCT_ID, APP_URL, SUPPORT_URL, RESEND_FROM, CHAT_RATE_LIMIT_PER_MIN, CHAT_FREE_DAILY_LIMIT, CHAT_PRO_DAILY_LIMIT

8. Sécurité applicative

8.1 Mesures en place

Mesure	Mise en œuvre
Authentification serveur	Vérification systématique du jeton ; l'identifiant provient du jeton, jamais de la requête
Refus par défaut en production	La fonction de conversation répond 503 si la vérification d'identité n'est pas active
Cloisonnement des données	Règles Firestore par propriétaire ; refus total au niveau des lignes sur Supabase
Révocation des procédures	Droits d'exécution retirés aux rôles anonyme et authentifié, accordés au seul rôle de service
Limitation de débit et quota	Contrôle atomique en base, insensible aux appels concurrents
Intégrité des paiements	Vérification de la signature des événements sur le corps brut
Anti-énumération de comptes	La réinitialisation de mot de passe ne révèle pas l'existence d'une adresse
Protection des tâches planifiées	Jeton partagé exigé
Injection d'instructions	Vingt-trois motifs de détection appliqués aux personas personnalisés, avec nettoyage des balises
Confinement des secrets	Aucun secret dans le paquet client, vérifié par recherche dans les fichiers construits

8.2 Risques identifiés

Risque	Portée	Recommandation
Point d'entrée de contexte ouvert	Relais de requêtes arbitraires, absence de quota	Ajouter authentification et limitation de débit
Point d'entrée de diagnostic exposé	Divulgaration de l'état de configuration	Supprimer, conformément à la mention portée dans le code
Origines autorisées sans restriction	En-tête d'origine permissif sur tous les points d'entrée	Restreindre à une liste d'origines connues
Collections Firestore inutilisées	Collections déclarées et lisibles par tout compte authentifié	Retirer les règles devenues sans objet
Absence de chiffrement applicatif	Contenus stockés en clair chez les hébergeurs	Évaluer un chiffrement des champs les plus sensibles
Absence de tests automatisés	Aucune couverture de non-régression	Introduire des tests sur les fonctions pures à fort enjeu

9. Glossaire technique

Terme	Définition
Persona	Configuration de comportement du modèle, définie par un rôle, une structure de sortie imposée et un niveau de friction
Niveau de friction	Intensité contradictoire appliquée : doux, moyen ou extrême
Marqueur	Balise textuelle insérée par le modèle dans sa réponse, interprétée puis retirée avant affichage
Carte de verdict	Représentation multidimensionnelle d'une vérification factuelle
Profil cognitif	Historique agrégé des biais et des réflexes de raisonnement observés chez un utilisateur
Steelman	Reformulation d'une thèse dans sa version la plus forte avant de la contester
Tolérance à la panne ouverte	Comportement consistant à laisser passer en cas de défaillance d'un contrôle non critique
Refus par défaut	Comportement consistant à refuser le service en cas de défaillance d'un contrôle critique
Drapeau fonctionnel	Clé activant une fonctionnalité pour une population restreinte d'utilisateurs

10. Historique du document

Version	Date	Objet
1.0	19 juillet 2026	Émission initiale